

MLA0201-FUNDAMENTALS OF MACHINE LEARNING

APPLICATION BASED QUESTIONS

CO1-ASSESSMENT TOOL -1

Student Name : D.Chinna Moulali **Reg no:**192425278

Q1) Implement The Concept Of Probability Distribution To Predict The Likelihood Of Spam Emails.

Aim

To implement the concept of probability distribution using the Naive Bayes algorithm for classifying emails as spam or non-spam.

Theory

Probability Distribution represents the likelihood of occurrence of different outcomes. In spam email detection, Naive Bayes uses Bayes' theorem to calculate the probability that an email belongs to the spam or ham category based on the occurrence of words. It assumes conditional independence among features, making it an efficient classifier for text classification.

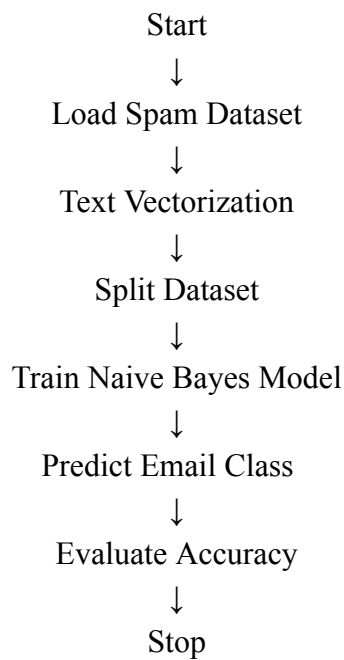
Dataset Used

- **Dataset Name:** Spam Email Dataset (spam.csv)
- **Source:** UCI Machine Learning Repository / Kaggle
- **Description:** Contains email messages labeled as Spam or Ham (Not Spam). The text is converted into numerical features using Count Vectorization.
- **Target Variable:** Label (Spam/Ham)

Algorithm

1. Import the required libraries.
2. Load the spam dataset.
3. Convert email text into numerical vectors.
4. Split the dataset into training and testing sets.
5. Train the Naive Bayes classifier.
6. Predict the class labels.
7. Calculate the accuracy.
8. Predict a new email.

Flowchart



Python Program

```
# =====  
# Spam Email Prediction using Naive Bayes  
# =====  
  
# Import Libraries import pandas as pd from  
sklearn.model_selection import train_test_split from  
sklearn.feature_extraction.text import CountVectorizer from  
sklearn.naive_bayes import MultinomialNB from sklearn.metrics  
import accuracy_score, classification_report  
  
# Load Dataset data = pd.read_csv("spam.csv",  
encoding="latin-1")  
  
# Display first five rows  
print("First 5 Rows:")  
print(data.head()) # Keep  
only required columns data =  
data[['v1', 'v2']]  
  
# Rename columns data.columns =  
['Label', 'Message'] # Convert  
labels into numbers data['Label'] =
```

```

data['Label'].map({'ham':0,
'spam':1})
# Features and Target
X = data['Message'] y
= data['Label']
# Convert text into numerical vectors vectorizer
= CountVectorizer()
X = vectorizer.fit_transform(X)
# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
X, y,
test_size=0.2,
random_state=42
)
# Train Naive Bayes Model model
= MultinomialNB()
model.fit(X_train, y_train)
# Predict prediction =
model.predict(X_test)
# Accuracy print("\nAccuracy:", accuracy_score(y_test,
prediction)) print("\nClassification Report")
print(classification_report(y_test, prediction))
# Predict New Email new_email
= [
"Congratulations! You have won a free iPhone. Click here to claim your reward." ]
new_email_vector = vectorizer.transform(new_email)
result = model.predict(new_email_vector)
print("\nNew Email Prediction:") if result[0] == 1:
print("Spam Email") else:
print("Not Spam")
Output

```

```

===== RESTART: C:/Users/shaik/OneDrive/Desktop/MLA0201/exp_email_spam.py =====
First 5 Rows:
   v1 ... Unnamed: 4
0  ham ...      NaN
1  ham ...      NaN
2  spam ...     NaN
3  ham ...      NaN
4  ham ...      NaN

[5 rows x 5 columns]

Accuracy: 0.97847533632287

Classification Report
      precision    recall  f1-score   support

     0       0.99      0.99      0.99       965
     1       0.91      0.93      0.92       150

 accuracy      0.98      0.98      0.98      1115
  macro avg       0.95      0.96      0.95      1115
weighted avg       0.98      0.98      0.98      1115

New Email Prediction:
Spam Email

```

Result

The Naive Bayes classifier successfully predicted whether an email was spam or nonspam using probability distribution concepts.

Q3) Design the steps of a learning system for student performance prediction.

Aim

To design and implement a machine learning system that predicts student performance using academic attributes.

Theory

A learning system extracts patterns from historical data and uses them to make predictions. In this experiment, a Decision Tree classifier learns from students' academic information such as study time, attendance, and previous grades to predict their performance.

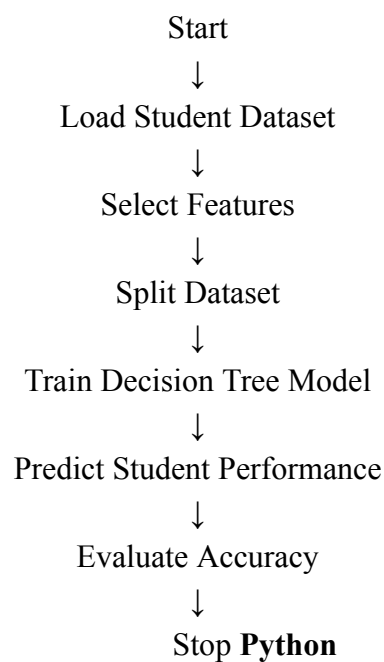
Dataset Used

- **Dataset Name:** Student Performance Dataset (student-mat.csv)
- **Source:** UCI Machine Learning Repository
- **Description:** Contains academic records of students including study time, failures, absences, and grades.
- **Target Variable:** Final Grade (G3) or Pass/Fail.

Algorithm

1. Import required libraries.
2. Load the student dataset.
3. Select input features and target.
4. Split the dataset.
5. Train the Decision Tree classifier.
6. Predict student performance.
7. Evaluate the model accuracy.

Flowchart



Program

```
# =====  
# Student Performance Prediction  
# =====  
# Import Libraries import pandas as pd from  
sklearn.model_selection import train_test_split from sklearn.tree  
import DecisionTreeClassifier from sklearn.metrics import  
accuracy_score, classification_report  
# Load Dataset data =  
pd.read_csv("student_data.csv", sep=",")  
# Display first five rows
```

```

print("First 5 Rows:") print(data.head())

# Select Features
X = data[['studytime', 'failures', 'absences', 'G1', 'G2']]

# Target Variable
# G3 >= 10 --> Good Performance
# G3 < 10 --> Poor Performance y
y = (data['G3'] >= 10).astype(int)

# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

# Train Decision Tree Model model =
DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)

# Predict prediction =
model.predict(X_test)

# Accuracy print("\nAccuracy:", accuracy_score(y_test,
prediction)) print("\nClassification Report")
print(classification_report(y_test, prediction)) # Predict
New Student new_student
= pd.DataFrame({
    'studytime':[3],
    'failures':[0],
    'absences':[2],
    'G1':[14],
    'G2':[15] }) result =
model.predict(new_student)
print("\nNew Student Prediction:")
if result[0] == 1:

    print("Good Performance") else:
    print("Poor Performance")

```

Output

```
===== RESTART: C:\Users\shail\OneDrive\Desktop\MLA8201\exp Student Performance.py =====
First 5 Rows:
  school sex  age address famsize Pstatus  ...  Walc  health absences  G1  G2  G3
0    GP   F   18     U    GT3     A  ...    1     3     6  5  6  6
1    GP   F   17     U    GT3     T  ...    1     3     4  5  5  6
2    GP   F   15     U    LE3     T  ...    3     3    10  7  8  10
3    GP   F   15     U    GT3     T  ...    1     5     2  15 14  15
4    GP   F   16     U    GT3     T  ...    2     5     4  4  10 10

[5 rows x 33 columns]

Accuracy: 0.8907341772151099

Classification Report
      precision    recall  f1-score   support

     0       0.85      0.85      0.85         27
     1       0.92      0.92      0.92         52

 accuracy      0.89
 macro avg      0.89      0.89      0.89         79
weighted avg      0.90      0.90      0.90         79

New Student Prediction:
Good Performance
|
```

Result

The Decision Tree classifier successfully predicted student performance based on academic features.

Q4) Develop a weather forecasting model using random variables and probability.

Aim

To develop a weather forecasting model using Gaussian Naive Bayes based on probability concepts.

Theory

Weather forecasting predicts future weather conditions using atmospheric measurements such as temperature, humidity, wind speed, and pressure. Gaussian Naive Bayes assumes that continuous features follow a Gaussian distribution and estimates the probability of each weather condition.

Dataset Used

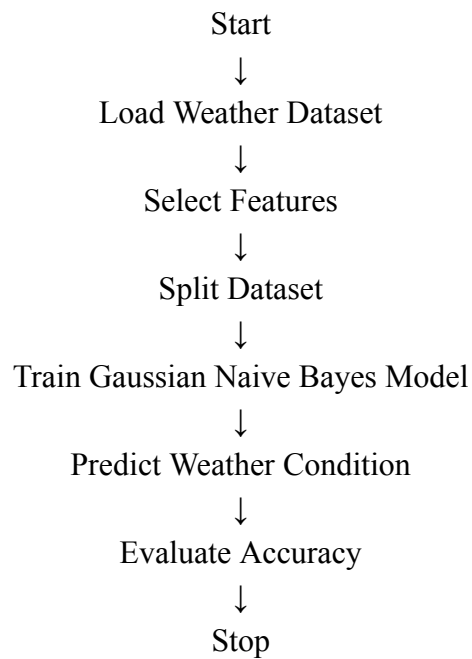
- **Dataset Name:** Weather History Dataset (weatherHistory.csv)
- **Source:** Kaggle
- **Description:** Contains historical weather observations including temperature, humidity, pressure, wind speed, visibility, and weather summaries.
- **Features Used:** Temperature, Humidity, Wind Speed, Pressure
- **Target Variable:** Summary

Algorithm

1. Import required libraries.

2. Load the weather dataset.
3. Select weather features.
4. Split the dataset.
5. Train the Gaussian Naive Bayes classifier.
6. Predict weather conditions.
7. Evaluate the model.

Flowchart



Python Program

```
# =====  
# Weather Forecasting using Probability Theory  
# =====  
# Import Libraries  
import pandas as pd from sklearn.model_selection import  
train_test_split from sklearn.naive_bayes import GaussianNB  
from sklearn.metrics import accuracy_score, classification_report
```



```

from sklearn.preprocessing import LabelEncoder

# Load Dataset data =
pd.read_csv("weatherHistory.csv")

print("First Five Rows")
print(data.head())

# Select Features
X = data[['Temperature (C)',
'Humidity',
'Wind Speed (km/h)',
'Pressure (millibars)']]

# Target Variable y =
data['Summary']

# Convert Text Labels into
Numbers encoder = LabelEncoder()
y = encoder.fit_transform(y)

# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
X, y,
test_size=0.2,
random_state=42
)

# Train Model model =
GaussianNB() model.fit(X_train,
y_train) # Prediction prediction =
model.predict(X_test)

# Accuracy print("\nAccuracy:", accuracy_score(y_test,
prediction)) print("\nClassification Report")
print(classification_report(y_test, prediction))

# Predict New Weather
new_weather = pd.DataFrame({
'Temperature (C)':[28],
'Humidity':[0.75],
'Wind Speed (km/h)':[12],

```

```

'Pressure (millibars)':[1015]
}) result = model.predict(new_weather)
print("\nPredicted Weather Condition:")
print(encoder.inverse_transform(result)
)

```

Output

```

=== RESTART: C:\Users\shaik\OneDrive\Desktop\MLA0201\exp Weather forecast.py ===
First Five Rows

```

	Formatted Date	...	Daily Summary
0	2006-04-01 00:00:00.000 +0200	...	Partly cloudy throughout the day.
1	2006-04-01 01:00:00.000 +0200	...	Partly cloudy throughout the day.
2	2006-04-01 02:00:00.000 +0200	...	Partly cloudy throughout the day.
3	2006-04-01 03:00:00.000 +0200	...	Partly cloudy throughout the day.
4	2006-04-01 04:00:00.000 +0200	...	Partly cloudy throughout the day.

```

[5 rows x 12 columns]

Accuracy: 0.4238764190555181

Classification Report

```

	precision	recall	f1-score	support
0	0.25	0.42	0.31	12
1	0.00	0.00	0.00	1
2	0.27	0.75	0.40	4
3	0.37	0.28	0.32	94
4	0.42	0.68	0.52	94
5	0.41	0.22	0.29	67
6	0.62	0.06	0.11	2183
8	0.00	0.00	0.00	8
9	0.00	0.00	0.00	3
10	0.00	0.00	0.00	3
11	0.04	0.41	0.07	17
12	0.40	0.78	0.53	1446
13	0.07	0.15	0.09	13
14	0.00	0.00	0.00	1
15	0.50	0.25	0.33	4
16	0.00	0.00	0.00	16
17	0.37	0.35	0.36	5618
18	0.36	0.34	0.35	3284
19	0.51	0.58	0.54	6392
20	0.00	0.00	0.00	0
21	0.00	0.00	0.00	0
23	0.00	0.00	0.00	1
24	0.30	0.75	0.43	8
25	0.20	0.67	0.31	6
26	0.67	0.12	0.21	16
accuracy			0.42	19291
macro avg	0.23	0.27	0.21	19291
weighted avg	0.45	0.42	0.40	19291

```
Predicted Weather Condition:
['Partly Cloudy']
|
```

Result

The Gaussian Naive Bayes classifier successfully predicted weather conditions using probability concepts.

Q6) Apply Conditional Probability concepts to design a fraud detection model for suspicious banking transactions.

Aim

To develop a fraud detection model using conditional probability and the Naive Bayes classifier.

Theory

Conditional Probability measures the probability of an event occurring given that another event has occurred. In fraud detection, it is used to estimate the likelihood that a transaction is fraudulent based on its transaction characteristics. Naive Bayes applies Bayes' theorem to classify transactions efficiently.

Dataset Used

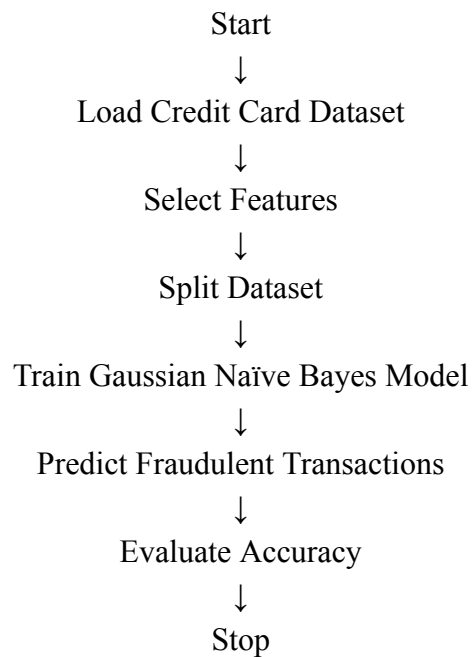
- **Dataset Name:** Credit Card Fraud Detection Dataset (creditcard.csv)

- **Source:** Kaggle
- **Description:** Contains credit card transactions made by European cardholders. Features V1 to V28 are anonymized for privacy, along with Time and Amount.
- **Target Variable:** Class ○ 0 – Genuine Transaction ○ 1 – Fraudulent Transaction

Algorithm

1. Import required libraries.
2. Load the credit card dataset.
3. Separate features and target.
4. Split the dataset.
5. Train the Gaussian Naïve Bayes classifier.
6. Predict fraudulent transactions.
7. Evaluate the model performance.

Flowchart



Python Program

=====

```

# Credit Card Fraud Detection using
# Conditional Probability (Naive Bayes)
# =====

# Import Libraries import pandas as pd from
sklearn.model_selection import train_test_split from
sklearn.naive_bayes import GaussianNB from sklearn.metrics
import accuracy_score, classification_report

# Load Dataset data =
pd.read_csv("creditcard.csv")
print("First Five Rows")
print(data.head())

# Select Features
X = data.drop('Class', axis=1)

# Target Variable
# 0 = Genuine #
# 1 = Fraud y =
data['Class']

# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
X, y,
test_size=0.2,
random_state=42
)

# Train Model model =
GaussianNB() model.fit(X_train,
y_train) # Prediction prediction =
model.predict(X_test)

# Accuracy print("\nAccuracy:", accuracy_score(y_test,
prediction)) print("\nClassification Report")
print(classification_report(y_test, prediction)) # Predict
New Transaction new_transaction = pd.DataFrame({
'Time':[50000],
'V1':[0.25],

```

```
'V2':[1.20],
'V3':[0.50],
'V4':[1.10],
'V5':[0.30],
'V6':[-0.10],
'V7':[0.20],
'V8':[0.05],
'V9':[0.60],
'V10':[0.15],
'V11':[0.40],
'V12':[0.80],
'V13':[0.20],
'V14':[-0.50],
'V15':[0.10],
'V16':[0.15],
'V17':[0.05],
'V18':[0.10],
'V19':[0.30],
'V20':[0.15],
'V21':[0.05],
'V22':[0.12],
'V23':[0.02],
'V24':[0.10],
'V25':[0.20],
'V26':[0.15],
'V27':[0.01],
'V28':[0.02],
'Amount':[250]
}) result =
model.predict(new_transaction)
print("\nTransaction Prediction:") if
result[0] == 1:
print("Fraudulent Transaction") else:
```

print("Genuine Transaction") **Output**

```
===== RESTART: C:\Users\shaik\OneDrive\Desktop\MLA0201\exp credit card.py =====
first five rows
  time    v1      v2      v3    ...    v27    v28  Amount  Class
0  0.0 -1.359807 -0.072781  2.536347 ...  0.133558 -0.021853  149.62    0
1  0.0  1.191857  0.266151  0.166480 ... -0.068983  0.014724   2.69    0
2  1.0 -1.358354 -1.340163  1.773209 ... -0.055353 -0.059752  378.66    0
3  1.0 -0.946272 -0.185226  1.792993 ...  0.062723  0.061458  123.50    0
4  2.0 -1.158233  0.877737  1.548718 ...  0.219422  0.215153   69.99    0

(5 rows x 31 columns)

Accuracy: 0.9930128857835048

Classification Report
      precision    recall  f1-score   support

     0       1.00      0.99      1.00     56864
     1       0.15      0.63      0.24        90

 accuracy      0.99      0.99      0.99     56962
 macro avg      0.57      0.81      0.62     56962
 weighted avg      1.00      0.99      1.00     56962

Transaction Prediction:
Genuine Transaction
|
```

Result

The Gaussian Naive Bayes classifier successfully identified genuine and fraudulent banking transactions using conditional probability.

Q7) Illustrate how Bayes Decision Theory can be applied in medical diagnosis for disease prediction.

Aim

To apply Bayes Decision Theory for predicting heart disease using patient medical information.

Theory

Bayes Decision Theory is a probabilistic approach that selects the class with the highest posterior probability. In healthcare, it helps predict whether a patient is likely to have heart disease based on clinical measurements such as age, blood pressure, cholesterol level, heart rate, and ECG results.

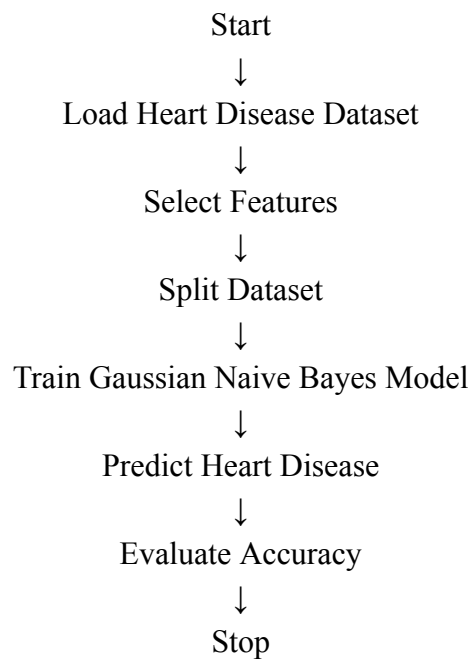
Dataset Used

- **Dataset Name:** Heart Disease Dataset (heart.csv)
- **Source:** UCI Machine Learning Repository / Kaggle
- **Description:** Contains clinical information of patients used for predicting the presence or absence of heart disease.
- **Features:** Age, Sex, Chest Pain Type, Resting Blood Pressure, Cholesterol, Fasting Blood Sugar, Resting ECG, Maximum Heart Rate, Exercise-Induced Angina, ST Depression, Slope, Number of Major Vessels, Thalassemia.
- **Target Variable:** Target ◦ 0 – No Heart Disease ◦ 1 – Heart Disease

Algorithm

1. Import required libraries.
2. Load the heart disease dataset.
3. Separate features and target.
4. Split the dataset.
5. Train the Gaussian Naive Bayes classifier.
6. Predict heart disease.
7. Evaluate the classifier.
8. Predict disease for a new patient.

Flowchart



Python Program

```
# Bayes Decision Theory for Medical Diagnosis import pandas as
pd from sklearn.model_selection import train_test_split from
sklearn.naive_bayes import GaussianNB from sklearn.metrics
import accuracy_score, classification_report
# Load dataset df =
pd.read_csv("heart.csv") #
Display first five rows
print("First Five Rows")
```



```

print(df.head())
# Features and Target X =
df.drop("target", axis=1) y =
df["target"]
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42
)
# Train Bayes Classifier model
= GaussianNB()
model.fit(X_train, y_train) #
Prediction y_pred =
model.predict(X_test)
# Accuracy print("\nAccuracy:",
accuracy_score(y_test, y_pred))
# Classification Report print("\nClassification Report\n")
print(classification_report(y_test, y_pred, zero_division=0))
# Predict new patient new_patient
= pd.DataFrame({
"age":[63],
"sex":[1],
"cp":[3],
"trestbps":[145],
"chol":[233],
"fbs":[1],
"restecg":[0],
"thalach":[150],
"exang":[0],
"oldpeak":[2.3],
"slope":[0],
"ca":[0],
"thal":[1]
}) prediction =

```

```

model.predict(new_patient)
print("\nMedical Diagnosis:") if
prediction[0] == 1:
print("Heart Disease Detected") else:
print("No Heart Disease")

```

Output

```

===== RESTART: C:\Users\shahk\OneDrive\Desktop\MLA0201\exp medical
First Five Rows
age sex cp trestbps chol fbs ... exang oldpeak slope ca thal target
0 52 1 0 125 212 0 ... 0 1.9 2 2 3 0
1 53 1 0 140 203 1 ... 1 3.1 0 0 3 0
2 70 1 0 145 174 0 ... 1 2.6 0 0 3 0
3 61 1 0 148 203 0 ... 0 8.9 2 1 3 0
4 62 0 0 130 294 1 ... 0 1.9 1 3 2 0

[5 rows x 14 columns]
Accuracy: 0.8
Classification Report

```

	precision	recall	f1-score	support
0	0.87	0.71	0.78	102
1	0.75	0.89	0.82	103
accuracy			0.80	205
macro avg	0.81	0.80	0.80	205
weighted avg	0.81	0.80	0.80	205

```

Medical diagnosis:
No heart disease

```

Result

The Gaussian Naive Bayes classifier successfully predicted the presence or absence of heart disease using Bayes Decision Theory.